When ViewModel Is a MUST

3 Critical Cases in Modern Application Architecture

Software Design Fundamentals · Best Practices · 2024

01 Data Persistence

02 Security

03 Separation of Logic

What is a ViewModel?

A ViewModel is an architectural component that sits between the View (UI) and the Model (data/business logic). It exposes data streams, commands, and state that the View can observe and react to — without the View ever needing to know where the data comes from or how it is transformed.

Key architectural benefits:

- !’ Lifecycle-aware data management
- !’ Cleaner, testable code separation
- !’ Consistent state across configuration changes
- !’ Improved security through data encapsulation

3 Cases Where ViewModel Is a Must

Detailed Analysis

01 Data Persistence Across Lifecycle Events

Configuration changes (e.g., screen rotation) destroy and recreate the UI layer

& Problem

When the user rotates the device or changes system settings, the Activity/Fragment is destroyed and recreated. Any data stored in the UI layer is lost, causing network calls to repeat and UI state to reset — degrading UX severely.

' ViewModel Solution

ViewModel survives configuration changes. Data is retained in the ViewModel's scope, so the UI simply re-subscribes to the same data streams on recreation without triggering new network requests or resetting state.

Ø=Ü¡ KEY POINT

```
// Data lives here:
class MyViewModel :
ViewModel() {
  val data =
MutableLiveData<List<Item>>()

  // Survives rotation!
  fun loadData() {
    // called ONCE
  }
}
```

& Critical for any app with async data loading

02 Security — Preventing Data Leaks & Unauthorized Access

Sensitive data exposed in the View layer becomes a serious security vulnerability

& Problem

Storing sensitive data (tokens, PII, payment info) directly in Activity/Fragment fields exposes it to memory dumps, logging, and improper lifecycle handling. UI callbacks can inadvertently leak credentials to logs or crash reports, violating GDPR, PCI-DSS compliance requirements.

' ViewModel Solution

ViewModel acts as a controlled data access layer. Raw sensitive data never reaches the View — only sanitized, masked, or display-safe representations are exposed. Access control, masking logic, and token management are centralized in the ViewModel, making audits and security patches easier.

Ø=Ü¡ KEY POINT

```
// NEVER in View:
// val token = rawApiToken

// ViewModel controls
exposure:
val maskedCard =
  _cardNumber.map {
    "**** **** **** " +
    it.takeLast(4)
  }
// Raw data NEVER leaves VM
```

Ø=Ý Mandatory for apps handling PII, tokens, or payment data

03 Separation of Business Logic from UI

Mixing business logic with UI code creates untestable, tightly coupled, fragile applications

& Problem

When business rules live inside Activities or Fragments, they become impossible to unit test without an Android emulator. Any UI change risks breaking business logic, and the same logic gets duplicated across screens, increasing bugs and maintenance significantly.

' ViewModel Solution

ViewModel cleanly separates what the app does (business logic, validation, state management) from how it looks (UI). The ViewModel can be unit-tested with plain JUnit — no Android dependencies needed. Multiple UI screens can share a single ViewModel, eliminating code duplication.

Ø=Ü¡ KEY POINT

```
// Pure, testable logic:
class CartViewModel :
ViewModel() {
  fun applyDiscount(code:
String)
    : Result<Discount> {
    // No UI dependency!
    // 100% unit-testable
  }
}
// Test without emulator '
```

& Essential for scalable, maintainable codebases

Case	Primary Benefit	Compliance / Standard	Priority
Data Persistence	No data loss on rotation	Material Design Guidelines	HIGH
Security	Prevents PII/token leaks	GDPR · PCI-DSS · HIPAA	CRITICAL
Logic Separation	Testable business rules	Clean Architecture / SOLID	HIGH